

EE789:Algorithmic Design of Digital Systems

Homework 1 Report

Name of student : Prathmesh Baral

Roll no : 23B1260

March 27, 2026

1 Question1

Write an Aa program that finds the maximum of four input numbers (each is \$uint< 32 >).

1.1 Answer

```
$module [maxoffour] $in (a b c d: $uint<32>) $out (e: $uint<32>) $is
{
    p := ( $mux (a > b) a b )
    q := ( $mux (c > d) c d )
    e := ( $mux (p > q) p q )
}
```

1.2 Simulation result

```
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# ./testbench_vhdl 10 20 5 15
Result=20
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# ./testbench_vhdl 1 2 3 4
Result=4
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# ./testbench_vhdl 7 3 9 2
Result=9
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# ./testbench_vhdl 99 23 99 1
Result=99
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# ./testbench_vhdl 12 45 7 30
Result=45
root@7a2dd2b6896f:/home/EE789_homework/maxofFour# |
```

Figure 1: Simulation result of maxoffour program

2 Question2

Write an Aa program that calculates the GCD of two numbers (each is \$uint< 32>).

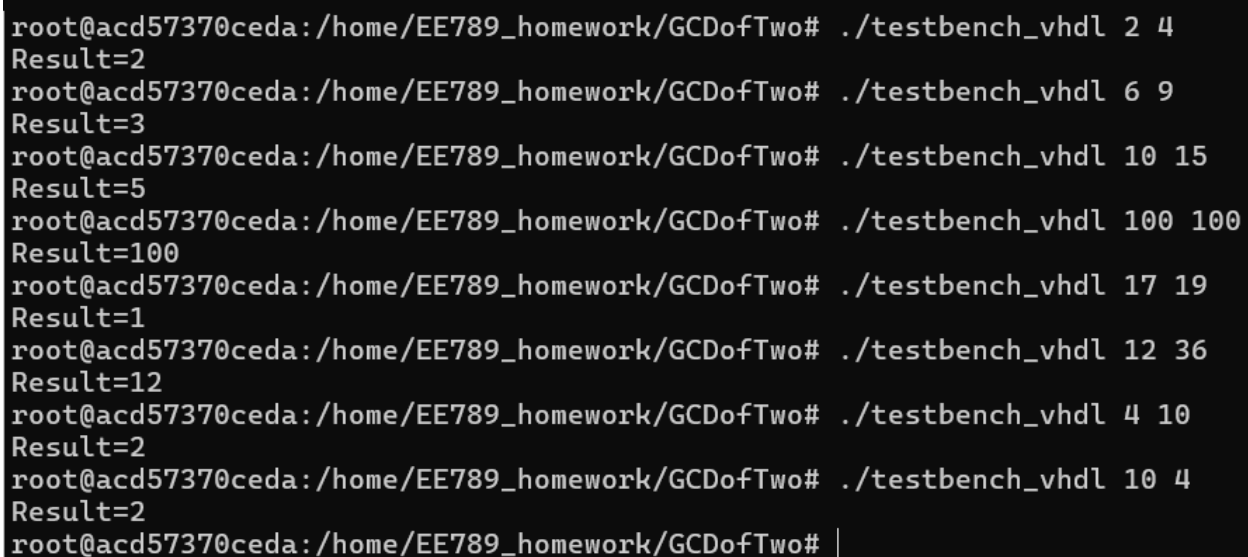
2.1 Answer

```
$module [gcdoftwo] $in (a b: $uint<32>) $out (y: $uint<32>) $is
{
    $branchblock[loop] {
        $merge $entry loopback
            $phi p := ( $mux (a > b) a b) $on $entry np $on loopback
            $phi q := ( $mux (a > b) b a) $on $entry nq $on loopback
        $endmerge
        c := (p - q)

        np := ( $mux (c > q) c q)
        nq := ( $mux (c > q) q c)

        $if ( p != q) $then
            $place[loopback]
        $else
            y := p
        $endif
    }
}
```

2.2 Simulation result



```
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 2 4
Result=2
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 6 9
Result=3
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 10 15
Result=5
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 100 100
Result=100
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 17 19
Result=1
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 12 36
Result=12
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 4 10
Result=2
root@acd57370ceda:/home/EE789_homework/GCDofTwo# ./testbench_vhdl 10 4
Result=2
root@acd57370ceda:/home/EE789_homework/GCDofTwo# |
```

Figure 2: Simulation result of gcdoftwo program

3 Question3

Write an Aa program that calculates the GCD of eight numbers (each is `$uint< 32 >`).

3.1 Answer

```
$module [gcdoftwo] $in (a b: $uint<32>) $out (y: $uint<32>) $is
{
    $branchblock[loop] {
        $merge $entry loopback
            $phi p := ( $mux (a > b) a b) $on $entry np $on loopback
            $phi q := ( $mux (a > b) b a) $on $entry nq $on loopback
        $endmerge
        c := (p - q)

        np := ( $mux (c > q) c q)
        nq := ( $mux (c > q) q c)

        $if ( p != q) $then
            $place[loopback]
        $else
            y := p
        $endif
    }
}

$module [gcdofeight] $in (a b c d e f g h: $uint<32>) $out (y: $uint<32>) $is
{
    $seriesblock
    {
        $parallelblock [p1]
        {
            $call gcdoftwo (a b) (p)
            $call gcdoftwo (c d) (q)
            $call gcdoftwo (e f) (r)
            $call gcdoftwo (g h) (s)
        } ( p => p q => q r => r s => s)

        $parallelblock [p2]
        {
            $call gcdoftwo (p q) (t)
            $call gcdoftwo (r s) (u)
        } ( t => t u => u)

        $call gcdoftwo (t u) (y)
    }
}
```

3.2 Simulation result

```
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 100 200 3
00 400 500 600 700 800
Result=100
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 1024 2048
4096 8192 16384 32768 65536 131072
Result=1024
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 6 10 14 2
2 26 34 38 46
Result=2
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 1 1 1 1 1
1 1 1
Result=1
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 2 4 6 8 1
0 12 14 16
Result=2
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 3 6 9 12
15 18 21 24
Result=3
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 12 18 24
30 36 42 48 54
ench_vhdl 8 12 16 20 24 28 32 36Result=6
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 8 12 16 2
0 24 28 32 36
Result=4
root@acd57370ceda:/home/EE789_homework/GCDofEight# ./testbench_vhdl 12 18 24
30 36 42 48 54
Result=6
root@acd57370ceda:/home/EE789_homework/GCDofEight# |
```

Figure 3: Simulation result of gcdofeight program

4 Question4

For the FIR filter shown below, draw the underlying Petri-net

```

$module [firFilter] $in (w0 w1 w2 w3: $int<8>)
  $out (c: $int<8>) $is
{
  $pipe in_data out_data: $uint<8>

  $branchblock[loop] {
    $merge $entry loopback
    $phi vals := $zero<24> $on $entry
    n_vals $on loopback
  }
  $endmerge
  x := in_data
  $volatile $split (vals 8 8 8) (v1 v2 v3)
  $call dot_product (w0 x w1 v1 w2 v2 w3 v3) (y)
  $volatile n_vals := ($concat x ($slice vals 23 8))
  out_data := y
  $place [loopback]
}
}

```

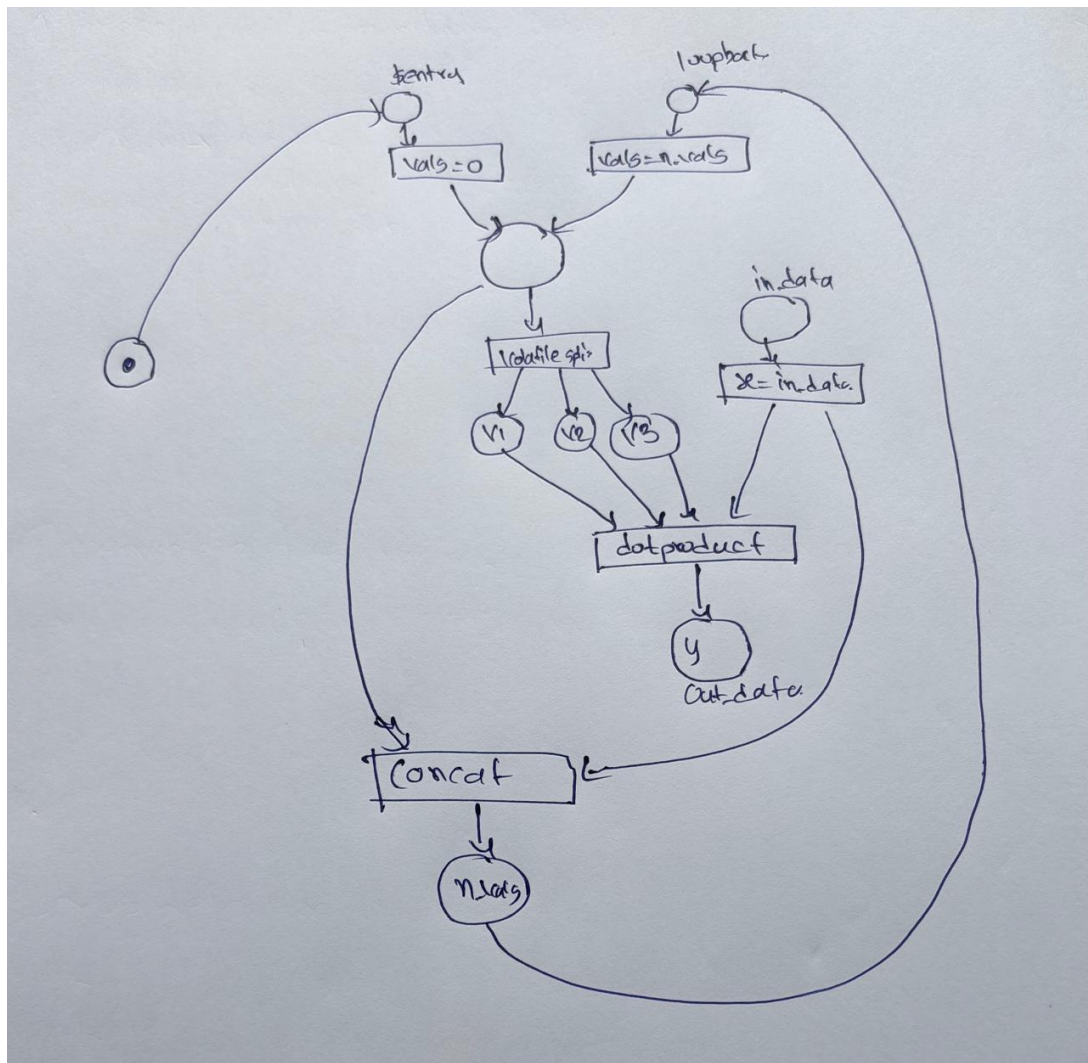


Figure 4: Petri net of FIR filter